Academic Portal

Comprehensive Technical Research Paper

Authors: Generated by Assistant
Date: 2026-05-17

Table of Contents

1. Abstract

This paper provides a thorough, technical examination of the *Academic Portal* web application. It documents architecture, code structure, data model, service responsibilities, control flows, and UI contracts (forms and JSON endpoints). The goal is to equip maintainers, auditors, and extension developers with an exhaustive reference to each page, action, field, and key interaction — plus concrete remediation steps for security and maintainability.

2. Introduction & Goals

- **Audience:** Backend & frontend developers, security reviewers, QA engineers, technical writers.
- **Scope:** Describe every user-visible page and underlying controller action(s); enumerate viewmodel properties and all form/input names; document DB interactions, transactions, and data integrity assumptions.
- **Deliverables:** Per-page descriptions, sequence diagrams (Mermaid), SQL examples, pseudocode for critical server-side transactions, recommendations and migration steps.

3. High-Level Architecture

The application is built on **ASP.NET Core MVC targeting .NET 8.0**. Requests are handled by controllers that render Razor views or return JSON for AJAX endpoints. The system is organized into four primary layers:

- **Presentation:** Razor views under *Views/* and shared partials (chatbot widget, admin sidebar, layout).
- **Controllers:** Coordinate request validation, call Services, and return views or JSON.
- **Services:** DataAccessService (ADO.NET wrapper), ChatbotService, AuditService.
- **Data:** SQL Server (scripts under *SQL/*), accessed through parameterized ADO.NET commands.
- **Middleware:** Logging, error handling, DB exception handling.

Request Lifecycle Example

1. Browser requests /Appointments/Book?facultyId=12&date=2026-05-18
2. **AppointmentsController.Book** (GET) returns a view with JS that calls GetAvailableSlots.
3. JS calls **AppointmentsController.GetAvailableSlots** (AJAX) → controller queries DB via DataAccessService.Query.
4. User submits booking form → **AppointmentsController.Book** (POST) performs transactional reservation using SqlTransaction.

4. Detailed Data Model (Schema + Examples)

The exact schema is defined in *SQL/init_schema.sql*. The following is an inferred schema with column names and recommended types, based on usage patterns in controllers and viewmodels.

```
CREATE TABLE Users (
  UserId       INT IDENTITY PRIMARY KEY,
  Username     NVARCHAR(100) NOT NULL UNIQUE,
  Email        NVARCHAR(200) NULL,
  PasswordHash NVARCHAR(256) NOT NULL,
  IsActive     BIT NOT NULL DEFAULT 1,
  CreatedAt    DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE TABLE Roles (
  RoleId INT IDENTITY PRIMARY KEY,
  Name   NVARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE UserRoles (
  UserId INT NOT NULL REFERENCES Users(UserId),
  RoleId INT NOT NULL REFERENCES Roles(RoleId),
  PRIMARY KEY(UserId, RoleId)
);

CREATE TABLE Faculty (
  FacultyId   INT IDENTITY PRIMARY KEY,
  UserId      INT NOT NULL REFERENCES Users(UserId),
  Designation NVARCHAR(200),
  Department  NVARCHAR(200),
  Office      NVARCHAR(100)
);

CREATE TABLE Students (
  StudentId        INT IDENTITY PRIMARY KEY,
  UserId           INT NOT NULL REFERENCES Users(UserId),
  FirstName        NVARCHAR(100),
  LastName         NVARCHAR(100),
  EnrollmentNumber NVARCHAR(50)
);

CREATE TABLE Courses (
  CourseId    INT IDENTITY PRIMARY KEY,
  Code        NVARCHAR(50),
  Title       NVARCHAR(300),
  Description NVARCHAR(MAX),
  IsActive    BIT DEFAULT 1
);

CREATE TABLE Enrollments (
  EnrollmentId INT IDENTITY PRIMARY KEY,
  StudentId    INT NOT NULL REFERENCES Students(StudentId),
  CourseId     INT NOT NULL REFERENCES Courses(CourseId),
  EnrolledAt   DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE TABLE AppointmentSlots (
```

```
  SlotId      INT IDENTITY PRIMARY KEY,
  FacultyId   INT NOT NULL REFERENCES Faculty(FacultyId),
  StartUtc    DATETIME2 NOT NULL,
  EndUtc      DATETIME2 NOT NULL,
  IsAvailable BIT NOT NULL DEFAULT 1
);

CREATE TABLE Appointments (
  AppointmentId INT IDENTITY PRIMARY KEY,
  StudentId     INT NOT NULL REFERENCES Students(StudentId),
  FacultyId     INT NOT NULL REFERENCES Faculty(FacultyId),
  SlotId        INT NULL REFERENCES AppointmentSlots(SlotId),
  Title         NVARCHAR(300),
  Description   NVARCHAR(MAX),
  Status        NVARCHAR(50) NOT NULL DEFAULT 'Booked',
  CreatedAt     DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE TABLE Feedback (
  FeedbackId     INT IDENTITY PRIMARY KEY,
  CourseId       INT NOT NULL REFERENCES Courses(CourseId),
  StudentId      INT NULL REFERENCES Students(StudentId),
  Rating         INT NOT NULL,
  Comments       NVARCHAR(MAX),
  SentimentScore FLOAT NULL,
  CreatedAt      DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE TABLE Policies (
  PolicyId      INT IDENTITY PRIMARY KEY,
  Title         NVARCHAR(300) NOT NULL,
  Category      NVARCHAR(100),
  Content       NVARCHAR(MAX),
  EffectiveDate DATE NULL,
  IsActive      BIT NOT NULL DEFAULT 1
);

CREATE TABLE AuditLog (
  AuditId     INT IDENTITY PRIMARY KEY,
  PerformedBy NVARCHAR(200),
  ActionType  NVARCHAR(100),
  TableName   NVARCHAR(100),
  RecordId    NVARCHAR(100),
  Details     NVARCHAR(MAX),
  CreatedAt   DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);
```

- Use DATETIME2 for precise timestamps and timezone-agnostic storage (store UTC values).
- IsAvailable flags on slots are the canonical availability source.
- Appointments.SlotId may be NULL for walk-in/unslotted appointments; controllers handle both.

5. Services, Middleware & Data Access Patterns

DataAccessService

- **Query(sql, params)** → **DataTable** — for SELECT-style queries.
- **Execute(sql, params)** → **int** — for non-queries (INSERT/UPDATE/DELETE).
- **ExecuteScalar** — for single-value queries (e.g., counts, inserted ID retrieval).

Transactional Usage (Pseudocode)

```
using var conn = new SqlConnection(connectionString);
conn.Open();
using var tx = conn.BeginTransaction();
try {
    var cmd = new SqlCommand(
        "SELECT IsAvailable FROM AppointmentSlots WHERE SlotId=@slotId", conn, tx);
    cmd.Parameters.AddWithValue("@slotId", slotId);
    var isAvailable = (bool)cmd.ExecuteScalar();

    if (!isAvailable) throw new InvalidOperationException("Slot already booked");

    var insert = new SqlCommand(
        "INSERT INTO Appointments(...) OUTPUT INSERTED.AppointmentId VALUES(...)", conn, tx);
    var newId = (int)insert.ExecuteScalar();

    var upd = new SqlCommand(
        "UPDATE AppointmentSlots SET IsAvailable=0 WHERE SlotId=@slotId", conn, tx);
    upd.Parameters.AddWithValue("@slotId", slotId);
    upd.ExecuteNonQuery();

    tx.Commit();
}
catch { tx.Rollback(); throw; }
```

Other Services

- **DbExceptionHandlingMiddleware:** Wraps request execution, catches SQL-related exceptions, selects error output based on request type (JSON vs HTML), and logs a RequestId for correlation.
- **AuditService:** Invoked for important operations (user CRUD, critical course changes). Example: *AuditService.Log(User.Identity.Name, "CreateUser", "Users", newUserId.ToString(), detailsJson)*.
- **ChatbotService:** Heuristic-based router that maps user text to intents. Returns ChatbotResponseViewModel with cards, suggestions, RequireLogin flag, and optional RedirectUrl.

6. Authentication & Authorization (Detailed)

Login Flow

1. User posts Username + Password.
2. Controller computes SHA-256 hex of provided password and queries DB.
3. If hashes match, query roles for the user and build ClaimsPrincipal.
4. Call HttpContext.SignInAsync(cookieOptions, principal).

Security Deficiencies

- SHA-256 used unsalted: vulnerable to precomputed attacks and fast brute force.
- No evidence of account lockout or failed-attempt tracking.
- Cookie options require audit for HttpOnly, SecurePolicy, SameSite, and expiration.

Recommended Hardening Steps

1. Add AspNetCore.Identity or a PasswordHasher<TUser> wrapper.
2. On next successful login, if user uses legacy SHA-256, perform rehash and store migration marker.
3. Add FailedLoginCount and LockoutEnd columns; enforce lockouts.

Replacement Code Example (C#)

```
var hasher = new PasswordHasher<User>();

// When creating a user
user.PasswordHash = hasher.HashPassword(user, plainPassword);

// When validating login
var result = hasher.VerifyHashedPassword(user, user.PasswordHash, plainPassword);
if (result == PasswordVerificationResult.Success) { /* sign-in */ }
```

7. Full Page-by-Page Breakdown

Input names and viewmodel properties are taken from Razor view markup and controller usage patterns.

Account Controller

- GET /Account/Login — Returns LoginViewModel. Fields: Username, Password, RememberMe.
- POST /Account/Login — Computes SHA-256 hex, compares with DB, builds claims, sets sign-in cookie.
- POST /Account/Logout — Calls SignOutAsync and redirects to the home page.

Dashboard Controller

- GET /Dashboard/Index — Returns DashboardViewModel with counts. Admins see site-wide metrics; students see enrolled courses and upcoming appointments.

Courses Controller

- GET /Courses/Index — Returns list of CourseViewModel.
- GET /Courses/Detail/{id} — Returns CourseDetailViewModel with Enroll form (hidden courseId).
- POST /Courses/Enroll — Authorized student action; inserts Enrollments record if not already enrolled.

Chatbot Controller

• POST /Chatbot/SendMessage — Receives message string; calls ChatbotService.ProcessMessage; returns JSON.

• POST /Chatbot/StartBooking — Begins booking; sets RequireLogin=true if unauthenticated.

• POST /Chatbot/ConfirmBooking — Finalizes booking using same transactional flow as AppointmentsController.

Appointments Controller

• GET /Appointments/Book?facultyId={id}&date;={yyyy-MM-dd} — Shows booking page.

• GET /Appointments/GetAvailableSlots?facultyId={id}&date;={yyyy-MM-dd} — Returns JSON slot list.

• POST /Appointments/Book — Fields: facultyId, date, slotId, title, description. Performs transactional creation.

• GET /Appointments/Detail/{id} — Shows appointment details with conditional actions (Complete, Cancel, Feedback).

• POST /Appointments/Cancel — Sets Status='Cancelled' and restores slot availability in a transaction.

Faculty Controller

• GET /Faculty/Index — Lists faculty rows; admin-only Create/Edit/Delete links.

• GET /Faculty/Profile/{id} — Shows profile info and upcoming slots/appointments.

• POST /Faculty/Create — Fields: userId, designation, department, office.

Feedback Controller

• GET /Feedback/Submit?courseId={id} — Shows rating and comments textarea.

• POST /Feedback/Submit — Fields: courseId, rating, comments. Saves to Feedback table with SentimentScore.

Policies Controller

• GET /Policies/Index?q=term — Searches policies by title/content.

• GET /Policies/Detail/{id} — Displays policy; content may contain raw HTML.

• GET/POST /Policies/Edit — Admin-only. Fields: PolicyId, Title, Category, EffectiveDate, Content, IsActive.

Admin Controllers

• AdminController.Users — Lists users; provides CreateUser/EditUser views.

• AdminStudentsController.Create — Admin creates student records; replace default-password generation with email token flow.

Student Controller

• GET /Student/Profile — Shows StudentProfileViewModel with personal data and quick links.

• POST /Student/UpdateProfile — Updates profile fields with input sanitization and validation.

8. Critical Sequence Flows

Booking Flow

The booking flow uses a SQL transaction to prevent race conditions when two students attempt to book the same slot simultaneously:

```
sequenceDiagram
  UI->>AppController: GET /Appointments/Book (facultyId, date)
  AppController->>UI: Book view HTML with JS
  UI->>AppController: AJAX GET /GetAvailableSlots (facultyId, date)
  AppController->>Service: Query available slots
  Service->>DB: SELECT * FROM AppointmentSlots WHERE FacultyId=@fid
                AND StartUtc>=@date AND IsAvailable=1
  DB-->>AppController: DataTable rows
  AppController-->>UI: JSON slots

  UI->>AppController: POST /Appointments/Book (slotId, title, description)
  AppController->>DB: Begin Transaction
  AppController->>DB: SELECT IsAvailable FROM AppointmentSlots WHERE SlotId=@slotId
  DB-->>AppController: IsAvailable=1
  AppController->>DB: INSERT INTO Appointments(...) OUTPUT INSERTED.AppointmentId
  AppController->>DB: UPDATE AppointmentSlots SET IsAvailable=0 WHERE SlotId=@slotId
  AppController->>DB: Commit
  AppController-->>UI: Redirect to BookConfirmation
```

Login Flow

```
sequenceDiagram
  Browser->>AccountController: POST /Account/Login { username, password }
  AccountController->>AccountController: ComputeSha256Hex(password)
  AccountController->>DB: SELECT UserId, PasswordHash FROM Users
                          WHERE Username=@username AND IsActive=1
  DB-->>AccountController: row
  AccountController->>AccountController: compare hashes
  alt hashes match
    AccountController->>DB: SELECT RoleName FROM Roles JOIN UserRoles ...
    DB-->>AccountController: roles
    AccountController->>Browser: Set authentication cookie and redirect
  else hashes mismatch
    AccountController-->>Browser: show login error
  end
```

9. Client-Side Behaviors and UX Details

• **swal-action convention:** Forms decorated with class="swal-action" use swal-global.js to intercept submits and show confirmation or success messages as configured by data-swal-* attributes.

• **appointments.js:** Implements slot rendering and selection. Formats StartUtc/EndUtc using client timezone via new Date(utcString).toLocaleString().

• **Chatbot widget:** Single-page mini-app; persists recent suggestions in local storage; displays RequireLogin prompts when server indicates.

Accessibility Notes

- Ensure form controls have associated <label> elements (most views do).
- Add aria-live regions for chatbot and status messages to announce dynamic content.

10. Security Analysis and Mitigation Plan

1. **Password Storage**

   Current: SHA-256 hex string (fast, unsalted). Risk: fast hashing enables efficient brute-force; no salt means rainbow tables apply. Action: adopt PasswordHasher<TUser> (PBKDF2) or Argon2. Implement rolling re-hash on user login and a forced reset for accounts that cannot be migrated.

2. **Authentication Cookies**

   Ensure CookieOptions are configured with HttpOnly=true, SecurePolicy=Always, SameSite=Lax (or Strict if suitable), short expiry and sliding expiration as needed.

3. **SQL Injection**

   DataAccessService uses parameterized commands — this is good. Verify that no dynamic SQL string concatenation exists. Replace any string-interpolated SQL with parameterized calls.

4. **Stored XSS**

   Policies store raw HTML and use @Html.Raw. Restrict HTML editing to admin roles and sanitize HTML with a trusted library (e.g., Ganss.XSS.HtmlSanitizer).

5. **CSRF**

   Razor pages use @Html.AntiForgeryToken() in forms. Verify global antiforgery options are enabled and validated for JSON endpoints as applicable.

6. **Access Control**

   Ensure [Authorize] attributes are present on all controller actions that require login. For state-changing endpoints, validate server-side that the caller is authorized.

7. **Concurrency & Data Integrity**

   Use DB constraints (unique indexes), rowversion columns, or transactional checks to prevent race conditions. Unit/integration tests must simulate concurrent booking attempts.

8. **Logging & Privacy**

   Audit logs should never store plain-text passwords. Mask or redact sensitive fields in logs.

11. Testing, CI/CD and Deployment Recommendations

- Add unit test projects for Services and controller logic using xUnit or NUnit.

• Add integration tests that spin up the app in-memory (WebApplicationFactory) and run booking flow tests against a test DB instance.

• Add a CI pipeline (GitHub Actions) that runs dotnet test and dotnet build and executes static analyzers (Roslyn analyzers, Snyk or Dependabot for dependencies).

• For DB migrations use a controlled migration strategy (Flyway, DbUp, or EF Core migrations) and keep create_db_and_seed.ps1 for local dev only.

Example GitHub Actions Job

```
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup .NET
        uses: actions/setup-dotnet@v3
        with:
          dotnet-version: '8.0.x'
      - name: Restore
        run: dotnet restore
      - name: Build
        run: dotnet build --no-restore -p:UseAppHost=false
      - name: Test
        run: dotnet test --no-build
```

12. Appendix: File Map, Key SQL Snippets & Next Steps

File Map

Controllers: *AccountController.cs*, *HomeController.cs*, *DashboardController.cs*, *CoursesController.cs*, *ChatbotController.cs*, *AppointmentsController.cs*, *FacultyController.cs*, *FeedbackController.cs*, *PoliciesController.cs*, *AdminStudentsController.cs*, *AdminCoursesController.cs*, *AdminController.cs*, *StudentController.cs*

Services: *DataAccessService.cs*, *ChatbotService.cs*, *AuditService.cs*

Middleware: *DbExceptionHandlingMiddleware.cs*

Views: *Views/Account/**, *Views/Admin/**, *Views/Appointments/**, *Views/Chatbot/**, *Views/Courses/**, *Views/Dashboard/**, *Views/Faculty/**, *Views/Feedback/**, *Views/Policies/**, *Views/Shared/**

SQL: *init_schema.sql*, *create_db_and_seed.ps1*, *add_appointment_feedback.sql*

Key SQL Snippets

```
-- Check slot availability
SELECT IsAvailable FROM AppointmentSlots WHERE SlotId=@slotId;

-- Reserve appointment and get id
```

```
INSERT INTO Appointments (StudentId, FacultyId, SlotId, Title, Description)
OUTPUT INSERTED.AppointmentId
VALUES (@studentId, @facultyId, @slotId, @title, @description);

-- Mark slot as booked
UPDATE AppointmentSlots SET IsAvailable=0 WHERE SlotId=@slotId;
```

Suggested Immediate Next Steps

1. Replace SHA-256 with PasswordHasher<TUser>; add migration path for existing users.
2. Add DB constraints for unique enrollments and optionally a rowversion column for concurrency checks.
3. Add HTML sanitization for Policies.Content at save and before render.
4. Create integration tests that simulate 10+ concurrent booking attempts to validate transactional behavior.

This document is intended as a detailed, actionable technical reference for the Academic Portal application. For verbatim code excerpts or exact SQL from controllers, contact the development team.